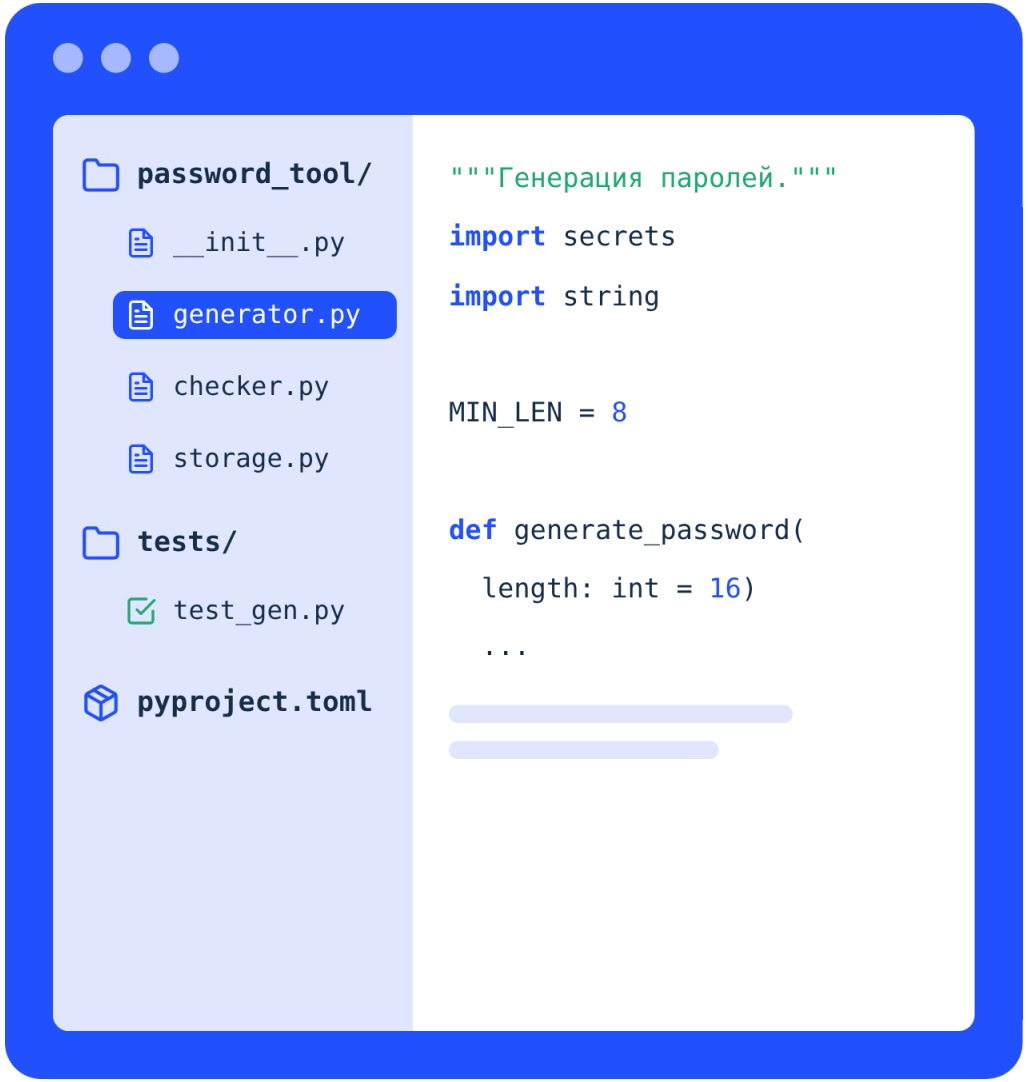
Лекция 8

Создание модулей

Структура, пакеты, документирование, тестирование, упаковка и безопасность модулей Python



Зачем создавать свои модули

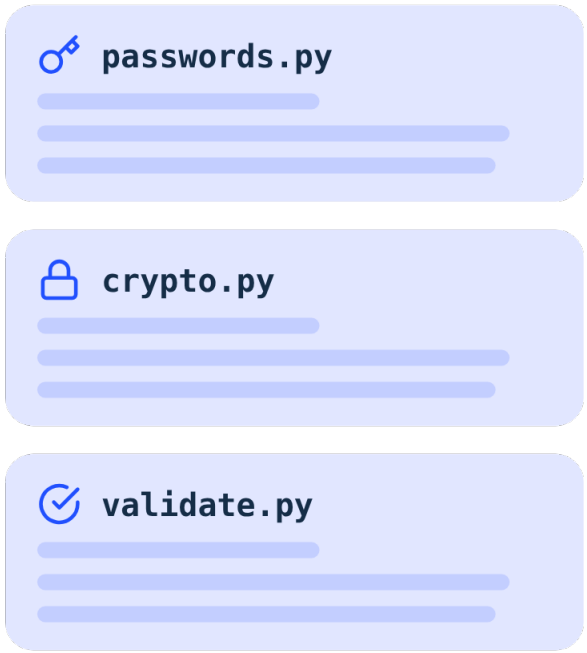
Рано или поздно готовых модулей не хватает — нужен свой, который можно переиспользовать или передать коллегам.

Любой проект из нескольких файлов — уже набор модулей

От их проектирования зависят **читаемость, тестируемость и безопасность**

main.py

1 файл, всё вперемешку



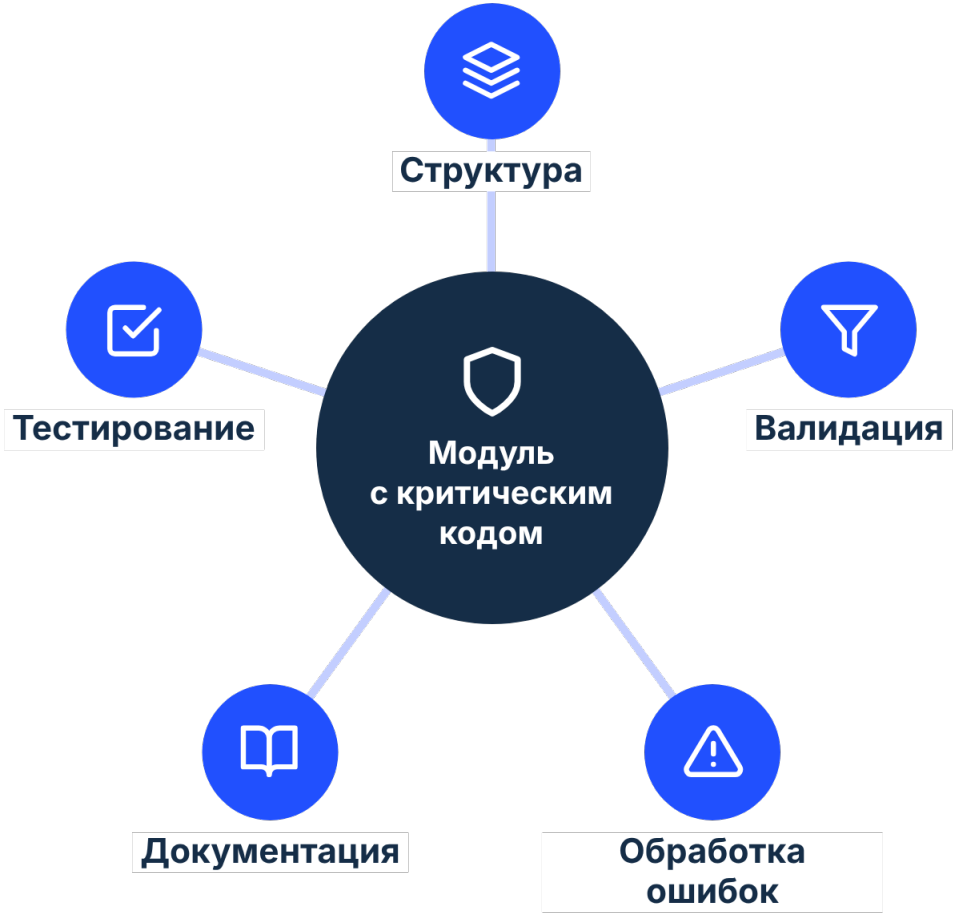
каждый модуль — одна задача

Модули в информационной безопасности

Модуль может содержать критический код: шифрование, аутентификацию, валидацию.

Ошибка в таком модуле — **уязвимость во всей системе**

Поэтому требования к нему повышены



Первый модуль: passwords.py

Создание модуля начинается с выделения логически связанного кода в отдельный файл.

Функции для работы с паролями → **passwords.py**

Модуль импортируется в другие файлы

Код перестаёт быть монолитным: появляются **повторное использование и изоляция**

passwords.py

```python
"""Модуль для работы с паролями."""
import secrets
import string

def generate_password(length=16):
    alphabet = string.ascii_letters
    ...

def check_length(password, min_length=8):
    return len(password) >= min_length
```



main.py

```python
from passwords import generate_password
print(generate_password(20))
```

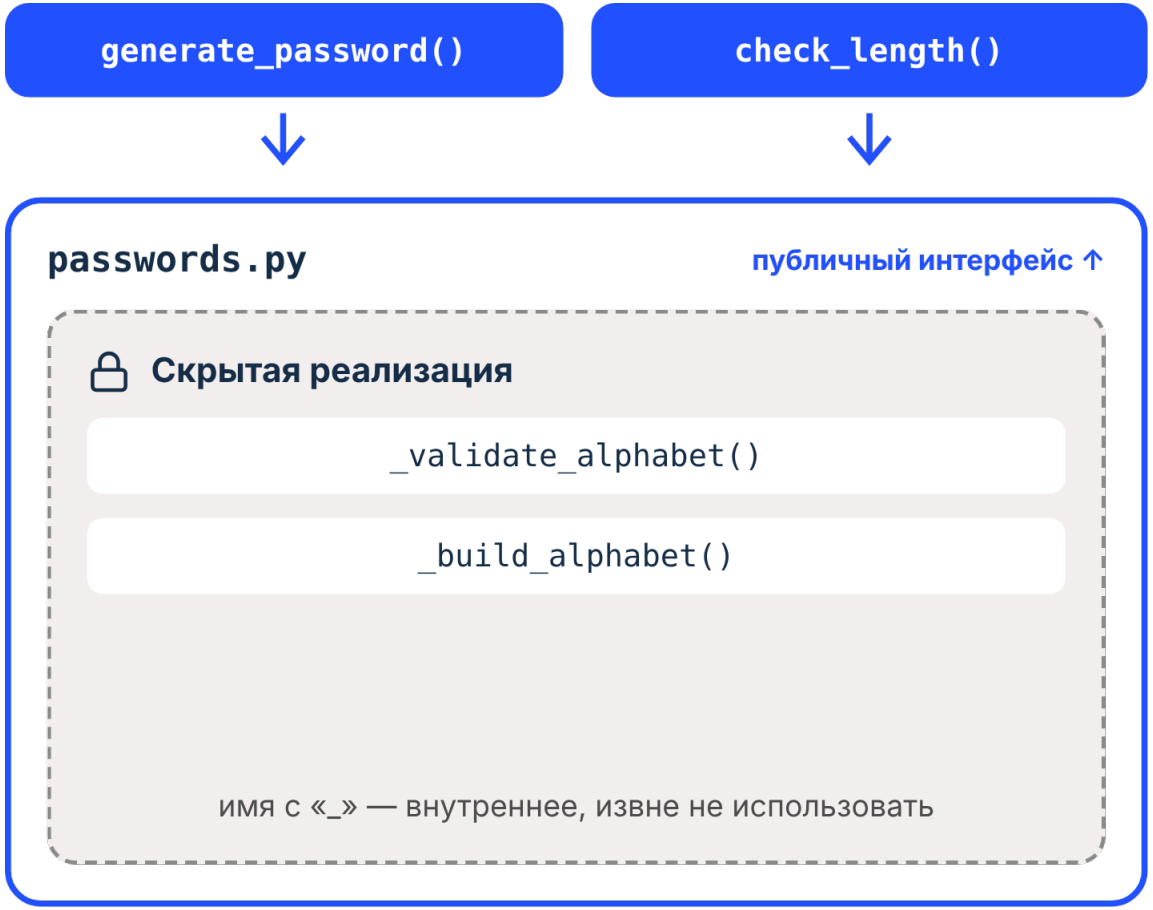
Публичный интерфейс и скрытая реализация

Просто вынести код в файл недостаточно — модуль нужно спроектировать.

Чёткое назначение

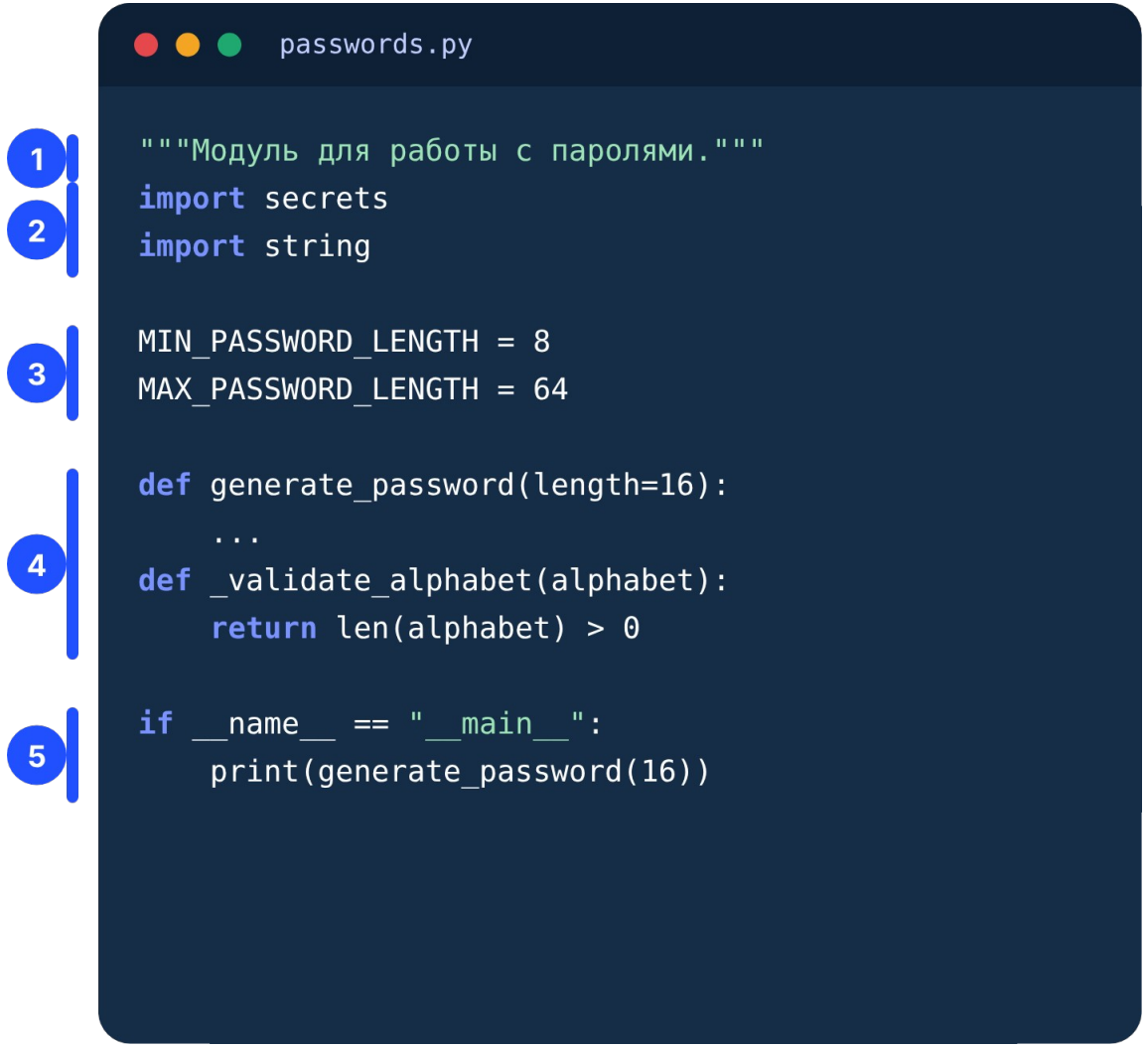
Публичный интерфейс для пользователей модуля

Имена с подчёркиванием (**_internal_function**) — внутренние, извне не используются



Структура модуля

1. **Docstring** — назначение модуля
2. **Импорты** — стандартная библиотека, сторонние, локальные
3. **Константы** — ВЕРХНИЙ_РЕГИСТР
4. **Классы и функции** — публичные, затем внутренние
5. **if __name__ == "__main__"** — запуск как программы



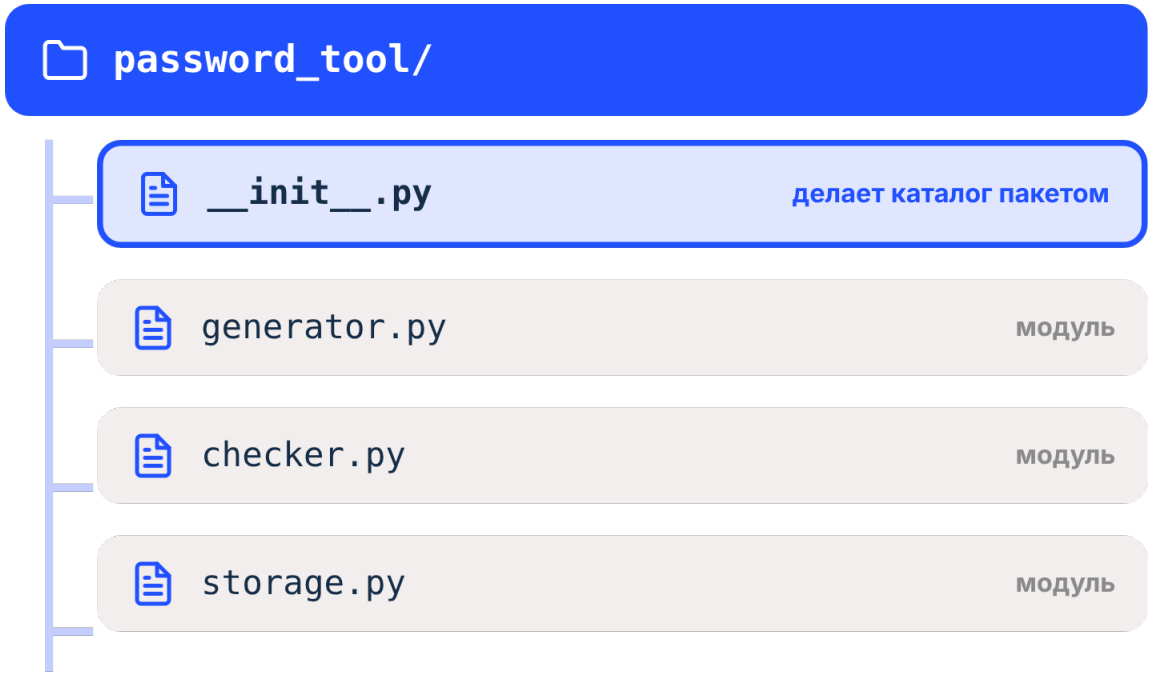
Пакеты

Когда модулей много, их объединяют в пакет — каталог с файлами .py и файлом **__init__.py**.

Иерархия модулей

Нет конфликтов имён

__init__.py может быть пустым



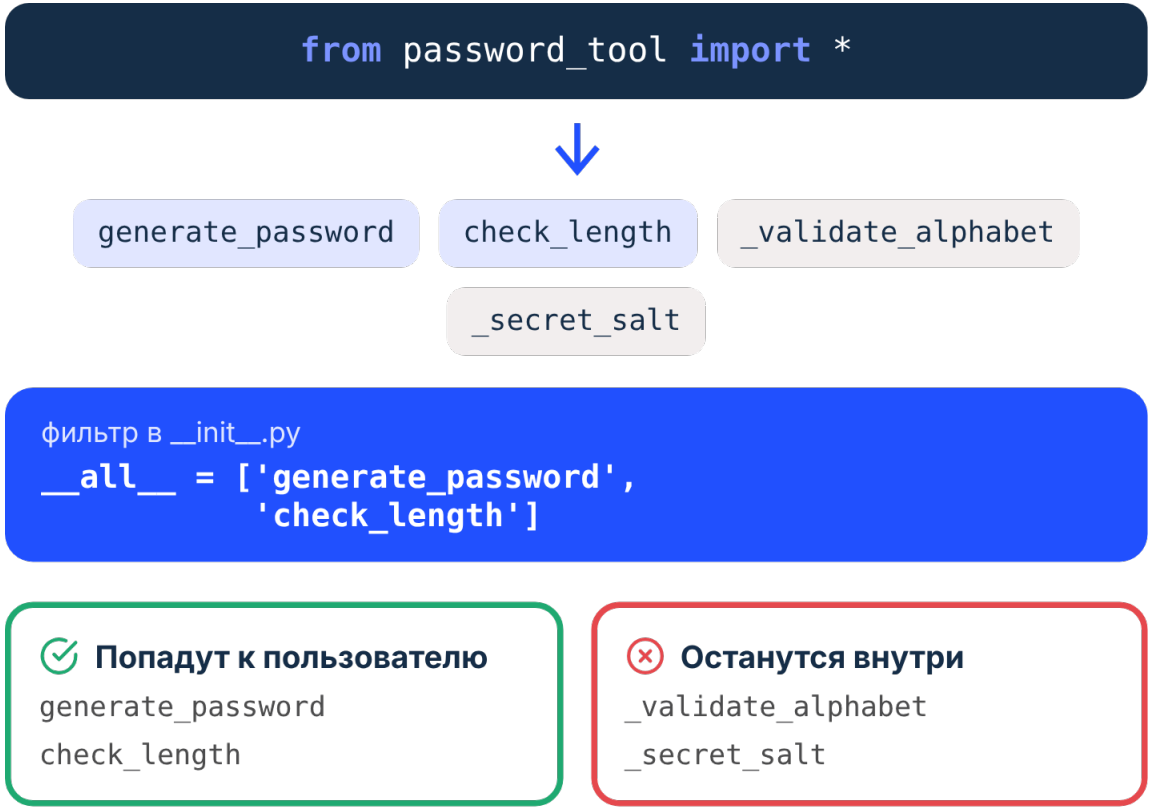

```
from password_tool.generator import ...
```

__init__.py и __all__

__init__.py определяет, какие модули экспортирует пакет.

__**all**__ явно задаёт имена для from password_tool import *

Внутренние функции не попадут в пространство имён пользователя



Подмена модуля

Имена своих модулей не должны совпадать со стандартными: secrets.py, socket.py.

Иначе Python импортирует ваш файл вместо стандартного

Злоумышленник может подменить модуль, который импортируется по имени

```
import secrets
```

Python ищет модуль по sys.path — по порядку:

1. Каталог проекта — **найден первым!**
   - `secrets.py` — **подмена**
2. Стандартная библиотека
   - `secrets.py` — настоящий

Защита: уникальные имена, абсолютные импорты, проверка целостности, ограничение sys.path

Документирование модуля

Docstring модуля отвечает на четыре вопроса.

Пример: модуль для генерации и проверки паролей, использует secrets, без спецсимволов

1

Что делает модуль?

2

Какие классы и функции предоставляет?

3

Какие зависимости использует?

4

Есть ли ограничения?

```
"""
Модуль для генерации и проверки паролей.
Использует secrets для стойкой генерации.
Ограничения: без спецсимволов.
"""
```

Документация — требование аудита

Аудитор должен видеть, что делает каждый модуль и какие меры защиты применяются.



Тестирование модулей

Каждый модуль тестируется в изоляции.

Используем pytest

Тесты проверяют то, чем будут пользоваться другие



Публичный интерфейс

тестируем то, что видят пользователи модуля, а не _внутренние функции



Граничные случаи

пустые значения, минимум и максимум, неверные типы

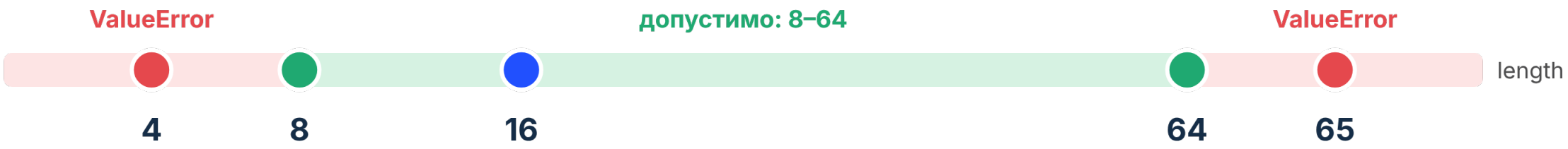


Изоляция

внешние зависимости заменяем заглушками unittest.mock

Граничные случаи на примере generate_password

Допустимая длина пароля — от 8 до 64. Проверяем границы, значения за ними и неверный тип.



4
меньше минимума
→ ValueError

8 и 64
ровно на границах
→ пароль создан

16
обычное значение
→ пароль создан

65
больше максимума
→ ValueError

"abc"
неверный тип
→ TypeError

Пример теста на pytest

Три теста проверяют длину пароля, ошибку при неверной длине и проверку длины.

pytest.raises проверяет, что выброшено исключение

tests/test_passwords.py

```python
import pytest
from password_tool.generator import generate_password

def test_generate_password_length():
    assert len(generate_password(16)) == 16

def test_generate_password_invalid_length():
    with pytest.raises(ValueError):
        generate_password(4)

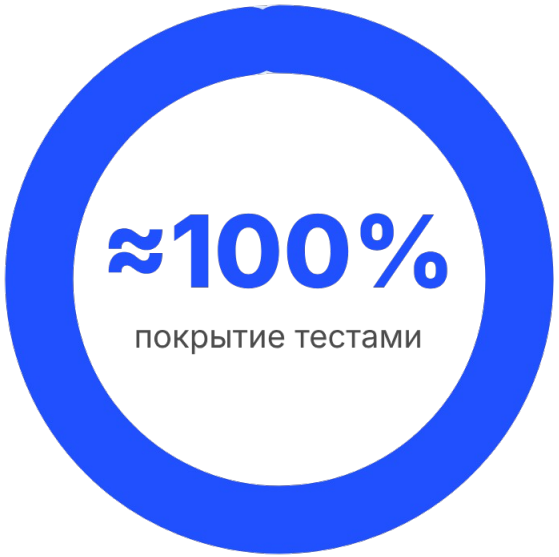
def test_check_length():
    assert check_length("abc", 8) is False
```

```
$ pytest
3 passed in 0.02s
```

Покрытие тестами

Модули безопасности должны иметь покрытие тестами, близкое к 100%.

Любая непротестированная ветка кода — **потенциальная уязвимость**



Шифрование

Аутентификация

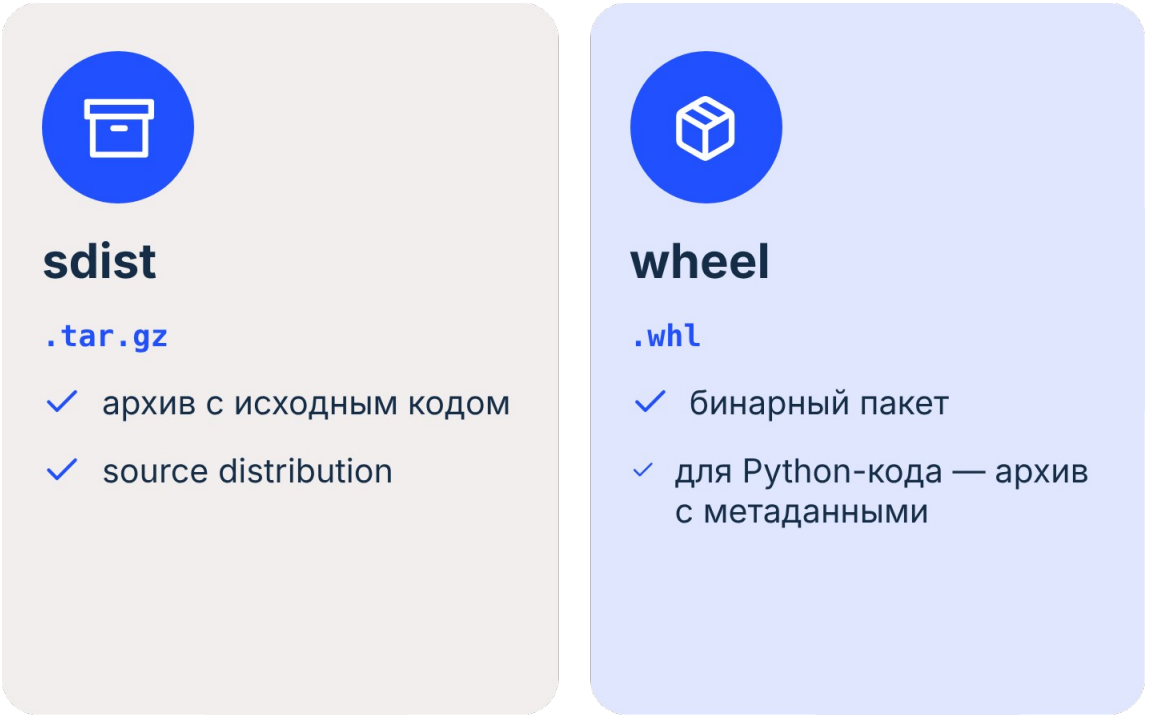
Валидация

Форматы распространения

Готовый модуль можно упаковать для распространения.

sdist — архив с исходным кодом

wheel — бинарный пакет; для Python-кода это просто архив с метаданными



pyproject.toml и сборка

pyproject.toml — современный стандарт описания пакета.

Имя, версия, описание, авторы

Зависимости с версиями

Сборка: **python -m build**

pyproject.toml

```
[project]
name = "password-tool"
version = "1.0.0"
dependencies = [
    "cryptography>=41.0.0",
]
[build-system]
requires = ["setuptools>=68.0"]
```



dist/

password_tool-1.0.0.tar.gz — sdist

password_tool-1.0.0-py3-none-any.whl — wheel

Безопасность упаковки

В пакет не должно попасть ничего лишнего, а сам пакет нельзя подменить.

Без секретов

не включать .env, ключи и тестовые данные

Проверка на секреты

```
grep -r "password"
dist/
```

Подпись пакета

GPG, Sigstore — защита от подмены

Доверенные репозитории

публикуем только туда

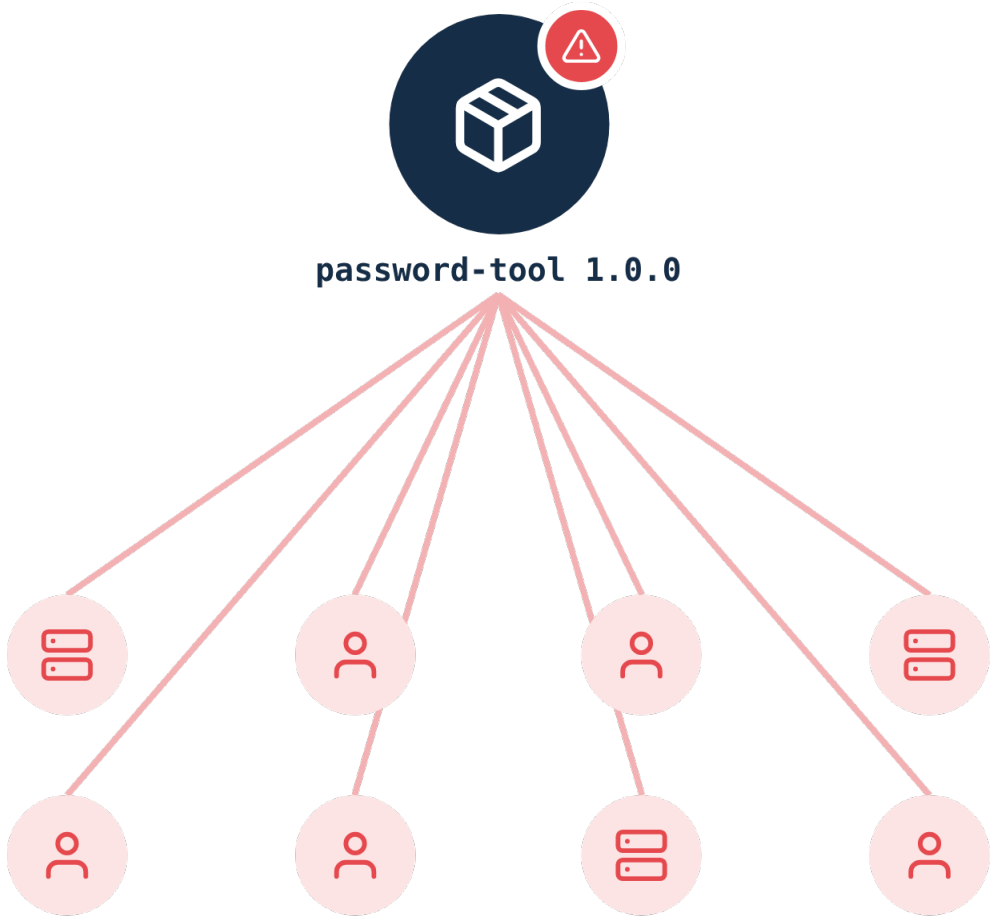
Аудит перед публикацией

`bandit · pip-audit`

Публикация в PyPI — ответственность

Уязвимость в опубликованном пакете затронет всех его пользователей.

Перед публикацией: аудит **bandit**, **pip-audit** и тестирование



Уязвимость в пакете → у всех, кто его установил

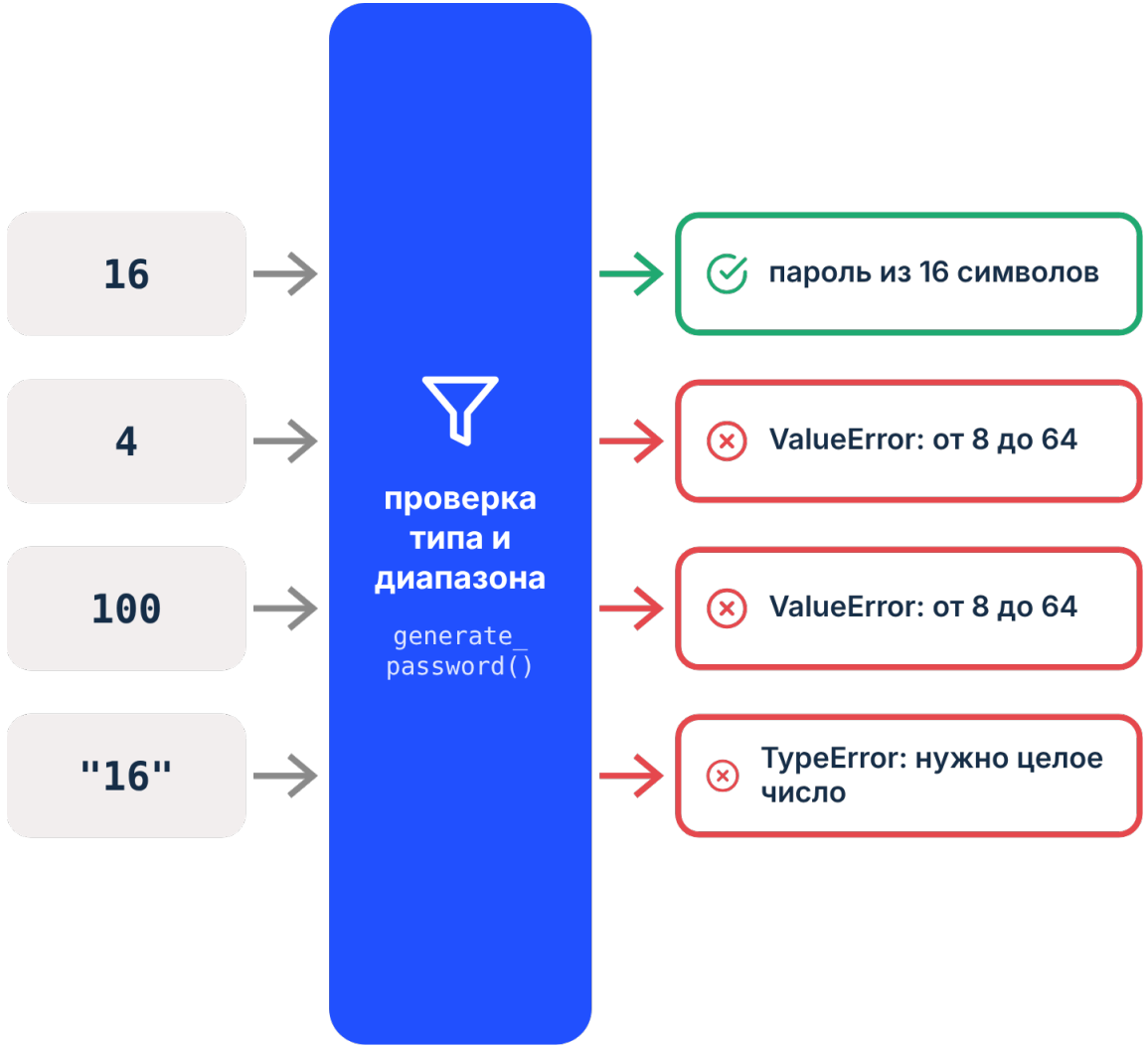
Валидация входных данных

Каждая публичная функция модуля проверяет входные данные.

Другие разработчики не знают, что передадут

Сначала тип — **TypeError**

Затем диапазон — **ValueError**



Сообщения об ошибках

Модуль не должен раскрывать внутреннюю информацию через ошибки.

Сообщение должно быть понятным

Без путей к файлам и имён таблиц БД

Неправильно

```
FileNotFoundError:
/home/admin/keys/master.key
table "users_secrets" not found
```

Раскрыты путь к ключу и имя таблицы БД

Правильно

```
ValueError:
Длина должна быть от 8 до 64
```

Понятно, что исправить, и ничего лишнего

Фиксация зависимостей

Сторонние библиотеки фиксируются в requirements.txt или pyproject.toml.

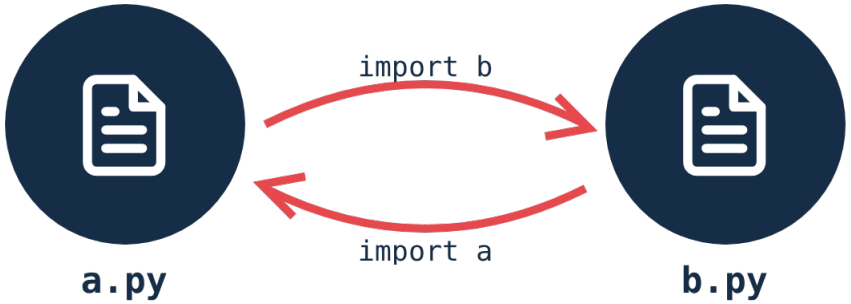
Нефиксированная версия = риск **внезапного изменения поведения**



Циклические импорты

Если модуль A импортирует B, а B — A, это сигнал о неправильной декомпозиции.

Не только техническая проблема, но и потенциальная уязвимость



Код труднее понять

Неожиданное поведение

Ошибка декомпозиции

Семантическое версионирование (SemVer)

Версия из трёх чисел сообщает, насколько серьёзны изменения.



MAJOR

несовместимые изменения API

`1.4.2 → 2.0.0`

MINOR

новые функции без поломки совместимости

`1.4.2 → 1.5.0`

PATCH

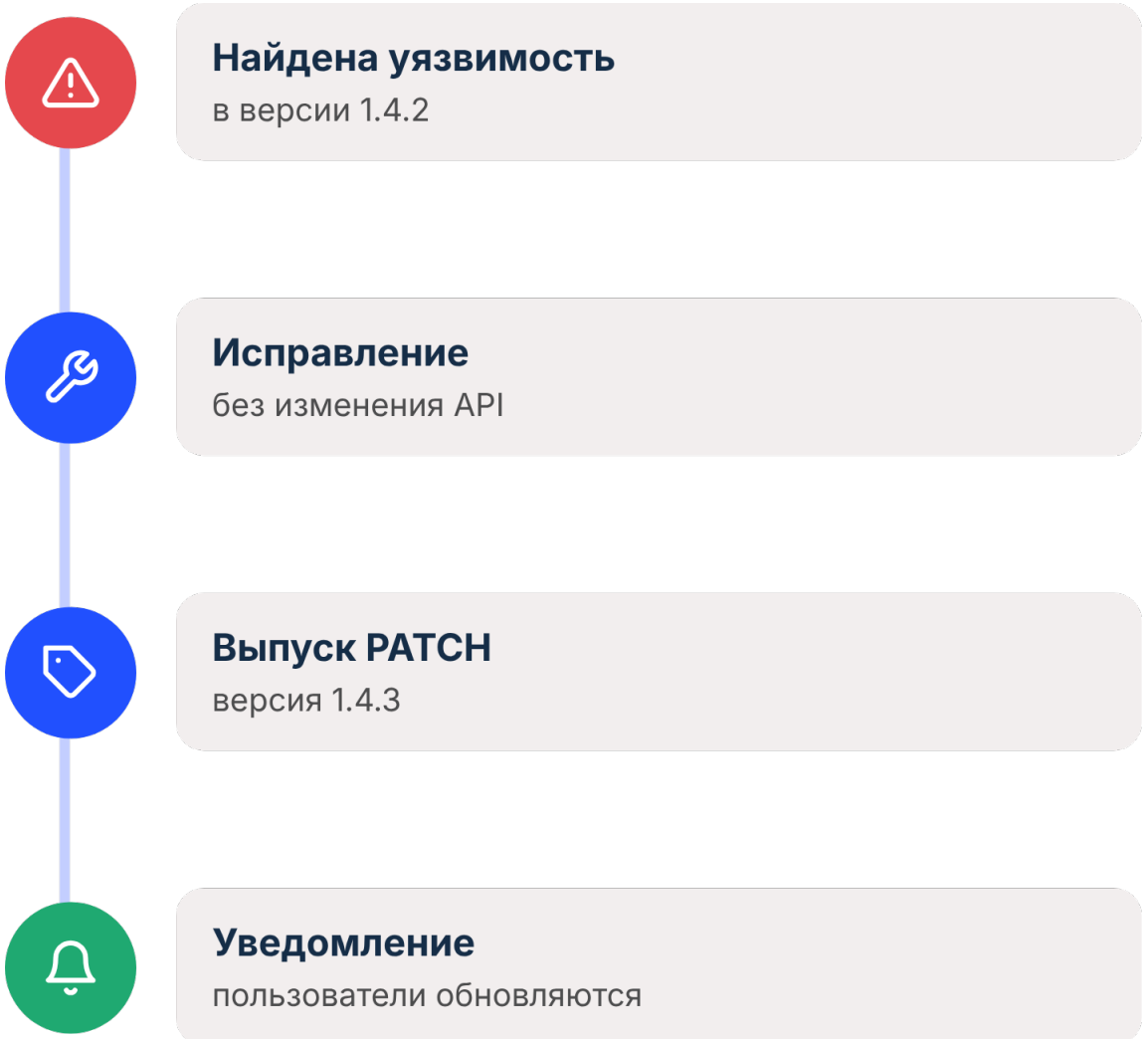
исправления ошибок и уязвимостей

`1.4.2 → 1.4.3`

Исправление уязвимости

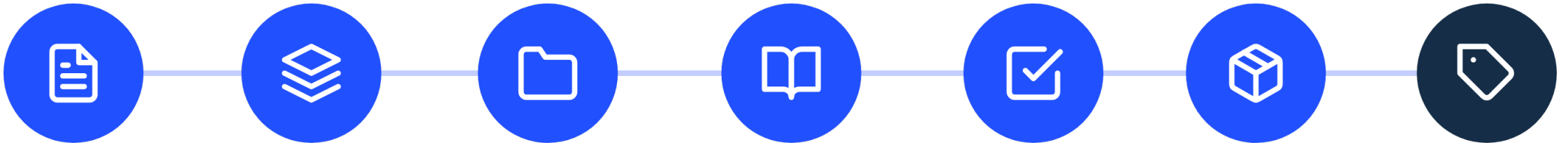
Исправили уязвимость — выпустите PATCH-версию и уведомите пользователей.

Это стандартная практика **ответственного раскрытия уязвимостей**



Заключение

Безопасный модуль — это структура, проверки, документация, тесты и аккуратная публикация.



01

Создать

логически связанный код — в файл

02

Структурировать

docstring, импорты, константы

03

Собрать в пакет

__init__.py и __all__

04

Документировать

docstrings, Sphinx

05

Тестировать

pytest, граничные случаи

06

Упаковать

pyproject.toml, без секретов

07

Версионировать

SemVer, PATCH при уязвимости